PROJECT REPORT

Data Synchronization Across Heterogeneous Systems

A Project Report Submitted

in Partial Fulfillment of the Requirements
for the Degree of

MASTER OF TECHNOLOGY
In

Computer Science & Engineering

By

Abhay Bhadouriya, Jaimin Jadvani, Jainish Parmar

(MT2024003, MT2024064, MT2024065)

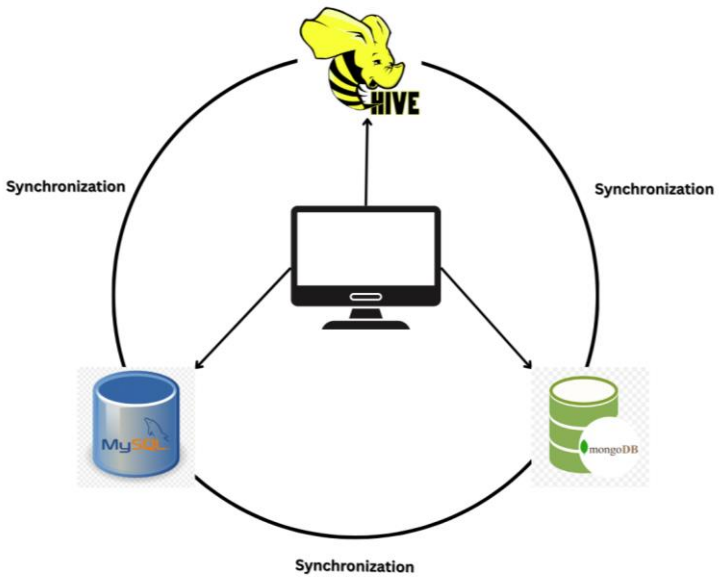
Department of Computer Science and Engineering
International Institute of Information Technology
Bangalore – 560100, India

April 2025

Table of Contents

Introduction

In this project, you are expected to demonstrate your understanding of data integration and synchronization across heterogeneous systems. We have integrated three different database systems: Hive, MongoDB, and MySQL.



We were required to implement methods for reading (get()) and updating (set()) data independently in each of these systems. Furthermore, each system should define an abstract function named merge(), which takes as an argument the name of another system whose state is to be merged with the calling system. The merge() function should not directly access the data from the other system; instead, it must rely solely on an operation log (oplog) that records the sequence of operations performed in that system. The operation log may be designed in a way that it is generic and applicable to any table, not just student course grades.csv.

Also, we were expected to design the merge functionality specific to each system – if necessary. For example, a command like PIG.MERGE(SQL) should merge the current state of the Pig table with the state of the PostgreSQL table (but not vice versa). The merge should be performed based on the operation logs of the two systems involved. These merge commands will be provided as input to your main program, which should execute them in the given order to synchronize the systems. Keep in mind the mathematical properties of

the merge operations, such associativity, commutativity, and idempotency, while implementing your merge function. Reflect on how convergence would occur in these scenarios, given the types of operations each system supports.

Hive

Hive is a data warehouse system built on top of Hadoop for managing and querying large datasets stored in distributed storage. It provides a SQL-like interface called HiveQL, allowing users to write queries without dealing directly with complex MapReduce programs. Hive is particularly useful for batch processing and analyzing big data. It translates SQL queries into MapReduce jobs, making it scalable and efficient for handling massive volumes of data. Hive stores metadata in a relational database like MySQL, while the actual data typically resides in Hadoop Distributed File System (HDFS). It supports various file formats such as ORC, Parquet, and Avro, making it flexible for different use cases. Though not ideal for real-time queries, Hive is excellent for data summarization, reporting, and data analysis tasks where high latency is acceptable. It is widely used in big data ecosystems and integrates well with tools like Apache Spark and Apache HBase.

MongoDB

MongoDB is a popular NoSQL database designed for handling large volumes of unstructured or semi-structured data. Instead of storing data in traditional tables and rows like relational databases, MongoDB uses flexible, JSON-like documents (called BSON) that can have different structures. This schema-less design allows for faster development and easier handling of evolving data models. MongoDB supports high availability with features like replication, and it can scale horizontally using sharding, making it suitable for big data and high-traffic applications. It is often used for real-time analytics, content management, IoT applications, and mobile apps. MongoDB provides a rich query language, indexing, and aggregation framework, and it integrates well with modern development stacks. Its flexibility, scalability, and ease of use have made it one of the most widely used NoSQL databases in cloud-native and distributed application architectures.

MySQL

MySQL is one of the most widely used open-source relational database management systems (RDBMS). It stores data in structured tables with predefined schemas and uses SQL (Structured Query Language) for accessing and managing the data. MySQL is known for its reliability, speed, and ease of use, making it a popular choice for web applications,

enterprise software, and data-driven services. It supports features like transactions, indexing, replication, and partitioning, ensuring data integrity and high availability. MySQL can handle small to large-scale applications and is often used with popular tech stacks like LAMP (Linux, Apache, MySQL, PHP/Python/Perl). It also offers robust security features and integrates well with cloud platforms. Owned by Oracle Corporation, MySQL has both a free community edition and commercial versions with additional features, making it versatile for different project needs. Its consistency and strong ecosystem have kept it a key player in database technologies for decades.

Implementation

OPLOG

An OPLOG (operations log) is a file or system that records a sequence of operations or changes made to data over time. It captures actions such as create, update, delete, or any other modification performed on a system's data. OPLOG are commonly used in systems that require data replication, synchronization, backup, or recovery. By keeping track of every change in the order it happened, an OPLOG allows other systems (like replicas or backups) to replay those operations and stay in sync with the original source. OPLOG helps improve efficiency because only the changes — not the entire dataset — need to be transmitted or restored. They are an essential part of many distributed databases, file systems, and applications that need strong data consistency and fault tolerance.

```
1, SET (SID1033,CSE016, A, 2025-04-21T18:45:56.188557)
2, GET (SID1033,CSE016, None, 2025-04-21T18:45:56.883265)
3, SET (SID1033,CSE016, B, 2025-04-21T18:45:57.089041)
4, SET (SID1033,CSE016, B, 2025-04-21T18:45:56.889835)
5, SET (SID1033,CSE016, B, 2025-04-21T18:45:56.889835)
6, SET (SID1033,CSE016, A, 2025-04-21T18:45:57.956807)
7, SET (SID1033,CSE016, A, 2025-04-21T18:45:56.188557)
8, SET (SID1033,CSE016, B, 2025-04-21T18:45:58.786244)
9, SET (SID1033,CSE016, B, 2025-04-21T18:45:57.089041)
10, SET (SID1033,CSE016, B, 2025-04-21T18:45:56.889835)
11, SET (SID1403,CSE006, A, 2025-04-21T18:45:59.578911)
12, SET (SID1403,CSE006, A, 2025-04-21T18:45:56.887788)
13, SET (SID1403,CSE006, B, 2025-04-21T18:46:00.021043)
14, SET (SID1403,CSE006, B, 2025-04-21T18:45:56.898911)
```

Figure: OpLog File

CRUD Operation

CRUD stands for Create, Read, Update, and Delete. These are the four basic operations you can perform on data in a database or any persistent storage system. We have explored and implemented CRUD operation for each database system and maintained OPLOG file. For each system there is a separate OPLOG file.

Flow of Crud Operation:

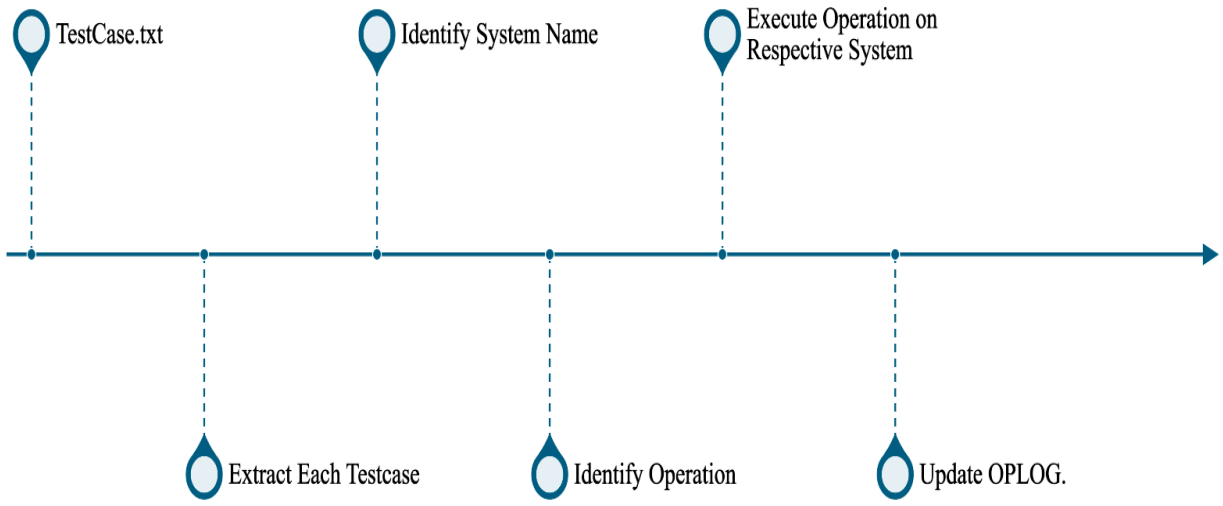
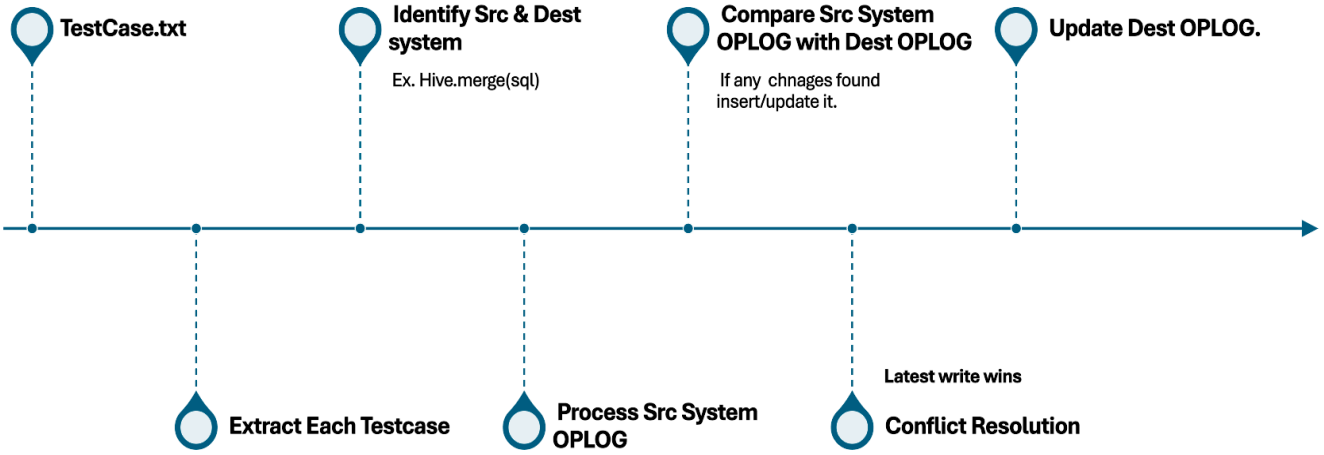


Figure: Flow of Crud Op.

MERGE Operation

Characteristic:

- One-way merge function.
  Ex. Command like HIVE.MERGE(SQL) should merge the current state of the HIVE table with the State of the MySQL table (but not vice versa).
- Commutativity: **YES**
  Ex. HIVE.MERGE(SQL) == SQL.MERGE(HIVE)
- Associativity: **YES**
  Ex. HIVE.MERGE(SQL.MERGE(MONGO)) ==(HIVE.MERGE(SQL)).MERGE(MONGO)
- Idempotency: **YES**
  Ex. HIVE.MERGE(HIVE)



Conflict Resolution:

A conflict happens during merging when the same record (usually identified by a key like a stu_id,course_id) is matched more than once meaning Database system does not know which version to apply.

Latest Writes Win is a conflict resolution approach where, if multiple updates occur on the same data, the version with the most recent timestamp is considered correct and is kept. Older updates are ignored. This strategy assumes that the latest change reflects the most accurate or intended state of the data.

Key Factor: **TIMESTAMP**

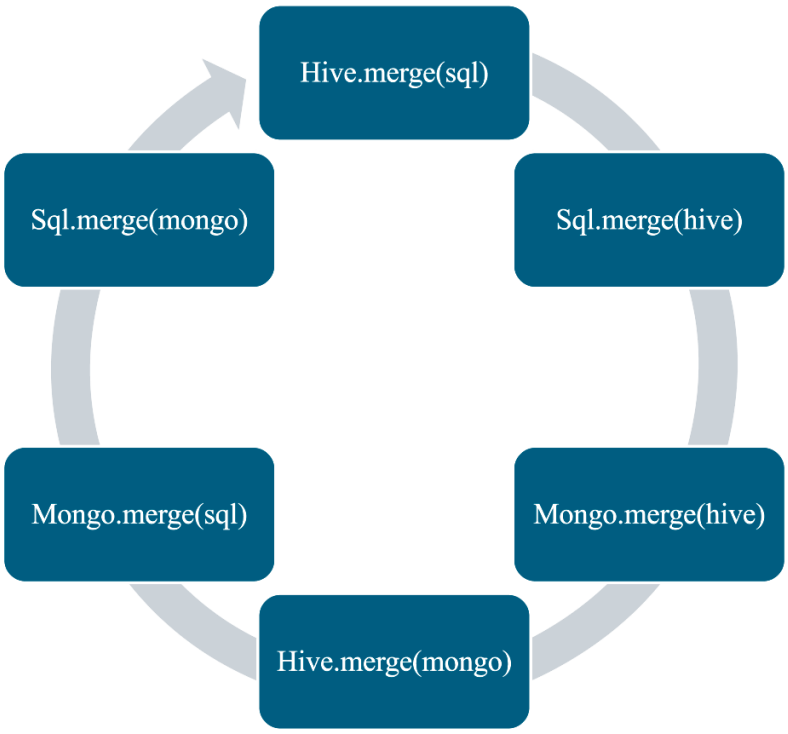
Results

The project has been successfully implemented, meeting all the defined requirements and objectives. Comprehensive testing was conducted using a variety of test cases covering normal and edge scenarios. The system demonstrated consistent and stable behavior across all tests. Functionality, performance, and reliability were thoroughly validated, and all components are working as expected.

Convergence

convergence means All three databases (MongoDB, MySQL, and the HIVE) eventually reach the same state meaning they all have the same set of rows, and for each row, the data matches exactly (based on the latest timestamp).

In ouf project, convergence requires either a two-way merge for each database pair or repeated one-way merges across all database pairs until the system reaches a stable state where no further updates occur.



Contribution Of Each Team Member

1) Abhay Bhadouriya (MT2024003)
   - **100/100**
2) Jaimin Jadvani (MT2024064)
   - **100/100**
3) Jainish Parmar (MT2024065)
   - **100/100**

Appendix – I

GITHUB PROJECT REPO LINK: CLICK HERE